

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC MÁY TÍNH
—————



BÀI TẬP LỚN MÔN HỌC
HỌC MÁY

CHỦ ĐỀ:
XÂY DỰNG PIPELINE HỌC MÁY TRUYỀN THỐNG
CHO DỮ LIỆU BẢNG/VĂN BẢN/ẢNH

Giáo viên hướng dẫn: TS. Trương Vĩnh Lân

Sinh viên thực hiện:

Mai Thanh Hải
Nguyễn Đức Việt

MSSV:

2033504
2115274

NHÓM: 03

TP. Hồ Chí Minh, Ngày 05 Tháng 12 Năm 2025

MỤC LỤC

I. Giới thiệu	3
1.1. Bối cảnh và mục tiêu bài toán.....	3
1.2. Bộ dữ liệu Adult/Census Income	3
II. Phân tích khám phá dữ liệu	3
2.1. Mô tả cấu trúc dữ liệu	3
2.2. Thống kê tổng quan	4
2.3. Phân phối các biến số.....	5
2.4. Tiền xử lý dữ liệu.....	8
III. Xây dựng mô hình	9
IV. Mô hình Deep Learning (tf.keras)	11
4.1. Chuẩn bị dữ liệu cho mô hình.....	11
4.2. Kiến trúc mạng Neural Network.....	11
4.3. Compile và cấu hình mô hình	12
4.4. Compile và cấu hình mô hình	12
4.5. Đánh giá mô hình trên tập kiểm tra	13
V. Đánh giá và so sánh mô hình	15
5.2. Phân tích chi tiết theo lớp:	15
5.3. Phân tích và so sánh:.....	15
5.3.1. Độ chính xác tổng thể:	15
5.3.2. Phân tích theo lớp:	16
5.3.3. Chỉ số tổng hợp:.....	16
5.4. Nhận xét về mất cân bằng dữ liệu:.....	16
VI. Kết luận.....	16
References.....	17
Dữ liệu (Datasets)	17
Bài viết/Hướng dẫn Trực tuyến	17

I. Giới thiệu

1.1. Bối cảnh và mục tiêu bài toán

Bối cảnh nghiên cứu xuất phát từ nhu cầu phân tích dữ liệu điều tra dân số Hoa Kỳ nhằm nhận diện những yếu tố nhân khẩu học và kinh tế – xã hội ảnh hưởng đến mức thu nhập cá nhân. Mục tiêu chính là xây dựng mô hình phân loại dự đoán liệu thu nhập hàng năm của một người có vượt quá ngưỡng 50.000 USD hay không, dựa trên thông tin về tuổi, trình độ học vấn, loại hình công việc, tình trạng hôn nhân và các chỉ số tài chính. Bộ dữ liệu Adult được lựa chọn vì đây là nguồn chuẩn mực do UCI cung cấp, có quy mô đủ lớn, cấu trúc đa dạng kết hợp giữa biến số và biến phân loại, giúp đánh giá hiệu quả của nhiều thuật toán học máy trên bài toán thu nhập cá nhân. Ý nghĩa thực tiễn thể hiện ở khả năng hỗ trợ hoạch định chính sách lao động, thiết kế chương trình đào tạo nghề, cũng như phát hiện các chênh lệch xã hội trong cơ hội việc làm và thu nhập.

1.2. Bộ dữ liệu Adult/Census Income

Bộ dữ liệu Adult (còn gọi là Census Income) từ UCI Machine Learning Repository là một trong những tập dữ liệu kinh điển để nghiên cứu bài toán dự đoán thu nhập cá nhân. Dữ liệu được trích xuất từ cuộc điều tra dân số Hoa Kỳ năm 1994, bao gồm thông tin nhân khẩu học, giáo dục, nghề nghiệp và các yếu tố kinh tế – xã hội khác.

Mục tiêu nghiên cứu là xây dựng mô hình phân loại nhị phân nhằm dự đoán liệu thu nhập hàng năm của một cá nhân có vượt quá ngưỡng 50.000 USD hay không. Biến mục tiêu income nhận hai giá trị “ $\leq 50K$ ” và “ $> 50K$ ”. Đây là một bài toán đặc trưng cho phân tích dữ liệu xã hội học và thường dùng để benchmark các thuật toán học máy trên dữ liệu dạng bảng.

II. Phân tích khám phá dữ liệu

2.1. Mô tả cấu trúc dữ liệu

Dataset có khoảng 32.561 mẫu huấn luyện (bản adult.data gốc) với 15 cột: 14 thuộc tính đầu là đặc trưng đầu vào, cột cuối là nhãn thu nhập

Các thuộc tính bao gồm cả số và phân loại, cụ thể:

- Thuộc tính số:

- age: tuổi
- fnlwgt: trọng số mẫu điều tra.
- education-num: số năm học (dạng số tương ứng với education).
- capital-gain: thu nhập tăng vốn.
- capital-loss: lỗ vốn.
- hours-per-week: số giờ làm việc mỗi tuần.

- Thuộc tính phân loại:

- workclass: loại hình công việc (Private, Self-emp-not-inc, Government,...).
- education: trình độ học vấn (Bachelors, HS-grad,...).
- marital-status: tình trạng hôn nhân (Married, Never-married,...).
- occupation: nghề nghiệp.
- relationship: vai trò trong gia đình (Husband, Wife, Not-in-family,...).

- race: chủng tộc.
- sex: giới tính.
- native-country: quốc gia xuất xứ.

- Biến mục tiêu:

- income: “<=50K” hoặc “>50K”, là nhãn cần dự đoán.

2.2. Thống kê tổng quan

Sau khi đọc dữ liệu, kết quả `df.shape` cho thấy bộ dữ liệu Adult bao gồm 32.561 quan sát và 15 cột, trong đó 14 cột là đặc trưng đầu vào và 1 cột là biến mục tiêu income. Thông tin từ `df.info()` xác nhận không có giá trị thiếu sau khi xử lý (tất cả 15 cột đều có 32.561 giá trị non-null), với 6 biến có kiểu dữ liệu số thực (float64) và 9 biến có kiểu số nguyên (int64), tổng dung lượng bộ nhớ sử dụng là 3.7 MB.

Các cột dữ liệu bị thiếu bao gồm:

- workclass với 1836 giá trị
- occupation với 1843 giá trị
- native-country với 583 giá trị

```
#Tổng quan
print(df.shape)           # số dòng, cột
print(df.info())          # kiểu dữ liệu, non-null
print("missing value each col: \n", df.isna().sum())  # số missing
mỗi cột
print(df.describe())      # thống kê cho các cột số
```

Bảng thống kê mô tả cho biến số:

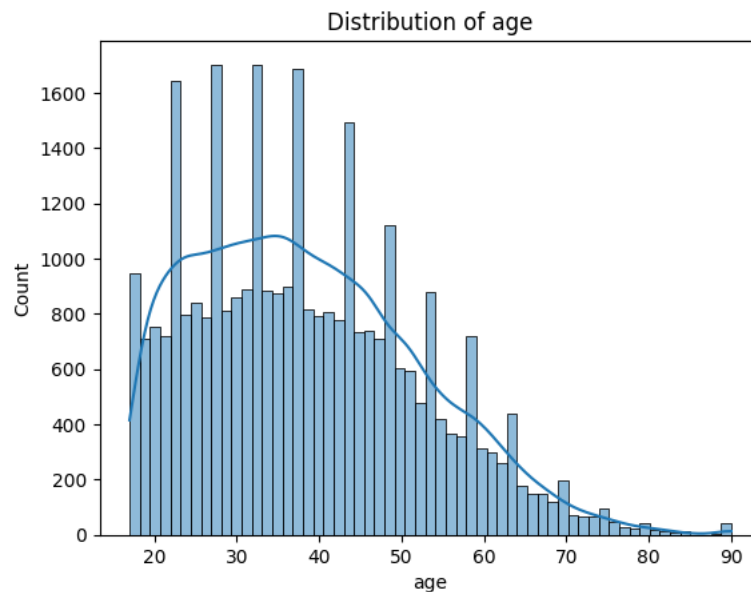
Thuộc tính	count	mean	std	min	25%	50%	75%	max
age	32561	38.58	13.64	17	28	37	48	90
fnlwgt	32561	189778.4	105550.0	12285	117827	178356	237051	1484705
education-num	32561	10.08	2.57	1	9	10	12	16
capital-gain	32561	1077.65	7385.29	0	0	0	0	99999

capital-loss	32561	87.30	402.96	0	0	0	0	4356
hours-per-week	32561	40.44	12.35	1	40	40	45	99

Từ bảng thống kê mô tả, ta có thể thấy:

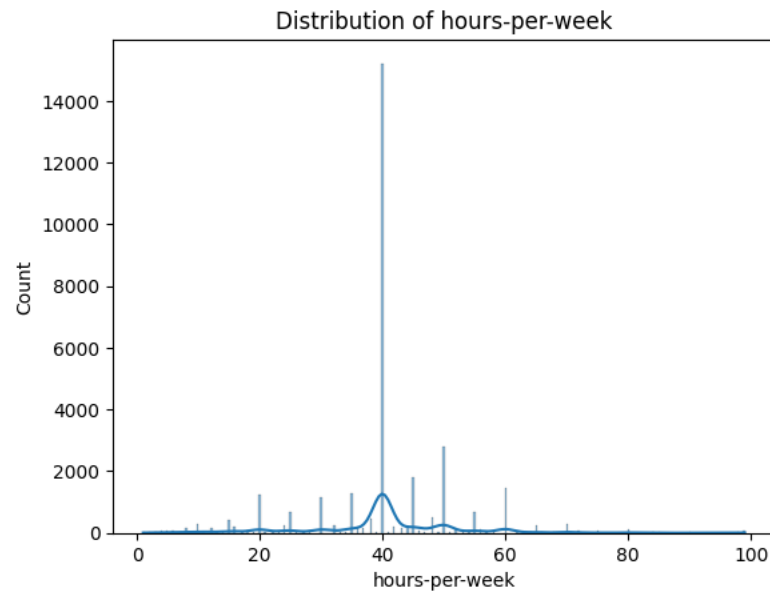
- **Phạm vi rộng của age:** Từ 17 đến 90 tuổi, với trung bình 38.58 và trung vị 37, cho thấy dữ liệu bao phủ nhiều nhóm tuổi. Phân phối tương đối đối xứng (trung vị gần trung bình), phù hợp với mô hình tuyến tính mà không cần biến đổi phức tạp.
- **Giá trị education-num:** Trung bình 10.08 năm, trung vị 10, tương đương trình độ trung học phổ thông đến một số năm đại học. Phạm vi từ 1 đến 16 năm phản ánh sự đa dạng về trình độ học vấn, là biến có khả năng ảnh hưởng mạnh đến thu nhập.
- **Phân phối lệch của capital-gain và capital-loss:** Cả hai biến đều có trung bình dương (1.077.65 và 87.30 USD) nhưng trung vị và các tứ phân vị đều bằng 0, cho thấy phần lớn quan sát không có khoản tăng/giảm vốn đáng kể. Chỉ một số ít có giá trị cao (max lần lượt là 99.999 và 4.356 USD), tạo ra phân phối lệch phải mạnh. Điều này gợi ý cần biến đổi logarit hoặc xử lý ngoại lai để cải thiện hiệu quả mô hình hồi quy logistic.
- **Phân phối tập trung của hours-per-week:** Trung bình 40.44 giờ, trung vị và tứ phân vị thứ nhất đều bằng 40, phản ánh phần lớn lao động làm việc toàn thời gian theo chuẩn 40 giờ/tuần. Tứ phân vị thứ ba là 45 giờ, cho thấy một bộ phận làm thêm giờ. Giá trị tối đa 99 giờ có thể là ngoại lai hoặc lỗi nhập liệu, cần kiểm tra và xử lý.

2.3. Phân phối các biến số



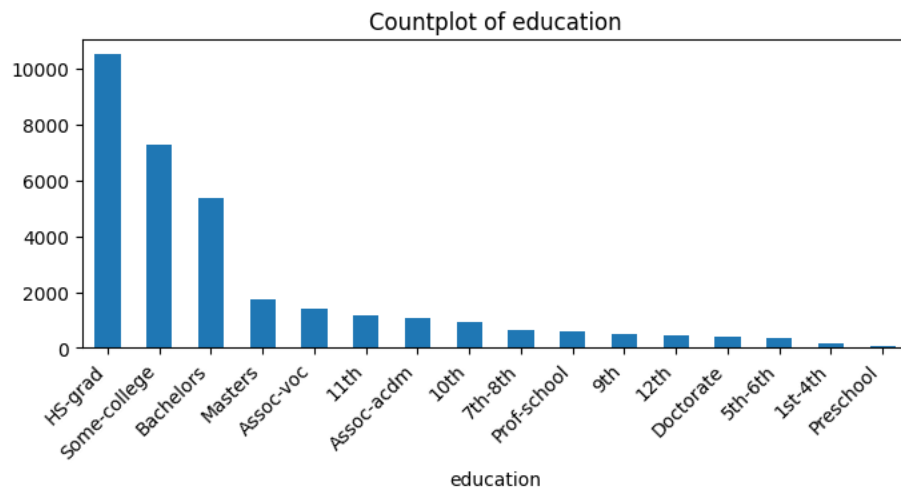
Hình 1

- **Phân phối biến age:** Histogram kèm KDE cho thấy phân phối lệch phải, đa mode với các đỉnh ở 20–40 tuổi. Phần lớn quan sát tập trung trong khoảng 20–45 tuổi, tần suất giảm dần sau 50 tuổi và rất thưa trên 70 tuổi. Phân phối này phù hợp với đặc điểm dân số lao động.



Hình 2

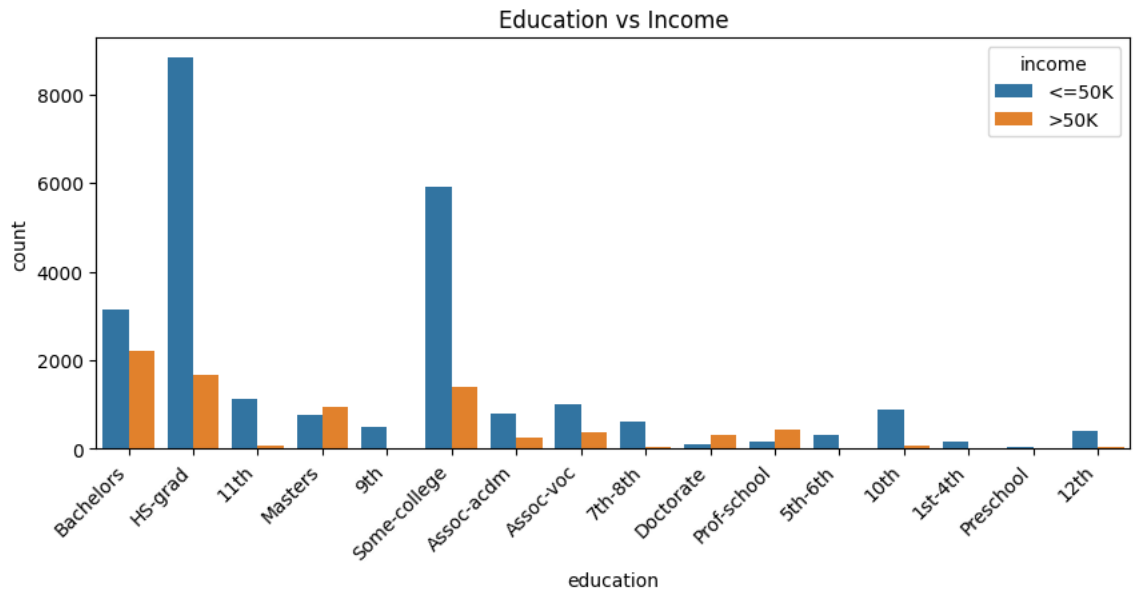
- **Phân phối biến hours-per-week:** Histogram kèm KDE cho thấy phân phối tập trung cực mạnh tại mức 40 giờ/tuần, thể hiện chuẩn làm việc phổ biến toàn thời gian. Các giá trị ngoài khoảng 30–50 giờ ít hơn đáng kể, và số quan sát ở mức rất thấp (<10 giờ) hoặc rất cao (>60 giờ, đến gần 99 giờ) khá thưa, cho thấy đây là các trường hợp làm việc bán thời gian hoặc làm thêm giờ đặc biệt.



Hình 3

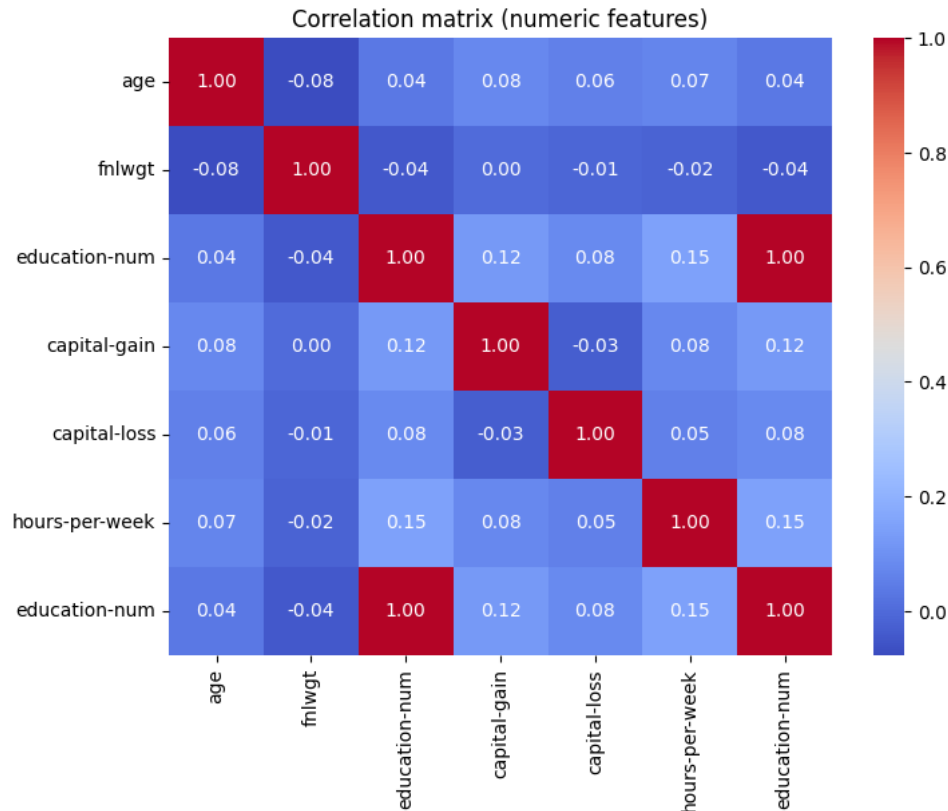
- **Phân bố education:** Countplot cho thấy hai nhóm HS-grad và Some-college chiếm đa số áp đảo trong dữ liệu, tiếp theo là Bachelors với số lượng thấp hơn đáng kể. Các bậc học văn cao hơn như Masters, Doctorate hay rất thấp như Preschool, 1st–4th xuất hiện với tần

suất rất ít, cho thấy phân bố trình độ học vấn bị lệch mạnh về nhóm tốt nghiệp trung học và một phần cao đẳng.



Hình 4

- **Mối quan hệ giữa education và income:** Biểu đồ cột cho thấy ở các mức học vấn thấp–trung bình như HS-grad và Some-college, số người có thu nhập <=50K chiếm ưu thế rõ rệt so với >50K. Ngược lại, ở các bậc học vấn cao như Bachelors, Masters, Doctorate, tỷ lệ người có thu nhập >50K tăng lên đáng kể so với nhóm <=50K, cho thấy trình độ học vấn cao có quan hệ thuận chiều với khả năng đạt thu nhập trên 50K.



Hình 5

- **Ma trận tương quan giữa các biến số:** education-num có tương quan dương nhẹ với hours-per-week và capital-gain, gợi ý số năm học cao thường đi kèm làm việc nhiều giờ hơn và có thu nhập từ vốn cao hơn; trong khi fnlwgt gần như không tương quan với các biến còn lại, phản ánh đây chủ yếu là trọng số mẫu điều tra hơn là đặc trưng nội tại của cá nhân.

2.4. Tiền xử lý dữ liệu

Trong bước tiền xử lý dữ liệu Adult, trước hết chuyển toàn bộ ký tự “?” xuất hiện trong các biến phân loại như workclass, occupation và native-country thành N/A để thuận tiện cho việc xử lý tiếp theo. Tùy theo tần suất xuất hiện, những quan sát có thiếu dữ liệu ít được giữ lại và điền giá trị phù hợp: mode đối với biến phân loại, median đối với biến số. Song song, làm sạch chuỗi văn bản bằng cách chuẩn hóa chữ viết, cắt bỏ khoảng trắng thừa để hạn chế phát sinh thêm các mức giả; sau đó áp dụng cơ chế mã hóa thích hợp (one-hot hoặc label) cho hầu hết biến phân loại.


```

# Xử lý missing value
# print(df.isna().sum())
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].median())
le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col])
#normalization number variables
scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])

```

Đoạn này thuộc phần Tiền xử lý dữ liệu, ngay sau EDA. Có thể viết/giải thích ngắn gọn trong báo cáo như sau:

- Xử lý giá trị thiếu: Với các biến phân loại (kiểu object), nhóm điền missing bằng mode của từng cột; với các biến số, điền bằng median để giảm ảnh hưởng outlier.
- Mã hóa biến phân loại: Sử dụng LabelEncoder để mã hóa các cột phân loại trong cat_cols thành giá trị số nguyên, giúp mô hình học máy xử lý được.
- Chuẩn hóa biến số: Áp dụng StandardScaler cho các cột số trong num_cols, đưa các biến về phân phối có trung bình 0 và độ lệch chuẩn 1, giúp mô hình tối ưu ổn định hơn.

III. Xây dựng mô hình

Sau khi tiền xử lý, dữ liệu được tách thành ma trận đặc trưng X (tất cả cột trừ income) và véc-tơ nhãn y (cột income đã mã hóa). Dữ liệu được chia thành tập huấn luyện và tập kiểm tra bằng train_test_split với tỷ lệ 80%–20% và random_state=42 để đảm bảo tái lập.

Hai mô hình được xây dựng và so sánh: Hồi quy Logistic và Random Forest. Hồi quy Logistic được khởi tạo với max_iter=1000 để đảm bảo hội tụ. Random Forest được cấu hình với 100 cây (n_estimators=100), không giới hạn độ sâu (max_depth=None), và random_state=42 để tái lập.

Quy trình đánh giá được thực hiện đồng nhất cho cả hai mô hình: huấn luyện trên tập huấn luyện, dự đoán trên tập kiểm tra, tính accuracy và in báo cáo phân loại. Ma trận nhầm lẫn được hiển thị bằng ConfusionMatrixDisplay để quan sát số lượng dự đoán đúng/sai theo từng lớp. Đường cong ROC được vẽ dựa trên xác suất dự đoán (predict_proba) hoặc điểm quyết định (decision_function), với AUC được tính để đánh giá khả năng phân biệt giữa hai lớp. Quy trình này cho phép so sánh hiệu suất của hai mô hình trên cùng một tập kiểm tra.

```

X = df.drop('income', axis=1)
y = df['income']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, auc
)
from sklearn.preprocessing import LabelBinarizer
import matplotlib.pyplot as plt

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        max_depth=None,
        random_state=42
    )
}

for name, clf in models.items():
    print("=" * 60)
    print(f"Model: {name}")
    clf.fit(X_train, y_train)

    # Dự đoán
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {acc:.4f}")
    print(classification_report(y_test, y_pred))

    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    plt.figure(figsize=(4, 4))
    disp.plot(cmap='Blues', values_format='d')
    plt.title(f"Confusion Matrix - {name}")
    plt.show()

```

```

# ROC curve
if hasattr(clf, "predict_proba"):
    y_score = clf.predict_proba(X_test)[: , 1]
else:
    y_score = clf.decision_function(X_test)

lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test).ravel()

fpr, tpr, _ = roc_curve(y_test_bin, y_score)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(5, 4))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve - {name}')
plt.legend(loc='lower right')
plt.show()

```

Cách tiếp cận này cho phép so sánh trực tiếp hiệu suất của hai mô hình trên cùng một tập kiểm tra, giúp lựa chọn mô hình phù hợp nhất cho bài toán dự đoán thu nhập.

IV. Mô hình Deep Learning (tf.keras)

4.1. Chuẩn bị dữ liệu cho mô hình

Trước khi đưa vào mạng neural, nhãn được chuyển đổi sang định dạng numpy array bằng `.values` để đảm bảo tương thích với TensorFlow/Keras. `y_train_dl` và `y_test_dl` được tạo từ `y_train` và `y_test` đã được mã hóa số từ bước tiền xử lý, sẵn sàng cho hàm mất mát `binary_crossentropy` và hàm kích hoạt `sigmoid` ở lớp đầu ra.

```

y_train_dl = y_train.values
y_test_dl  = y_test.values

```

4.2. Kiến trúc mạng Neural Network

Mô hình Deep Learning được xây dựng bằng Sequential của Keras với kiến trúc ba lớp:

- Lớp đầu vào: Input với kích thước bằng số đặc trưng của `X_train` (`input_dim = X_train.shape[1]`).
- Lớp ẩn thứ nhất: Dense với 64 nơ-ron, hàm kích hoạt ReLU, theo sau là Dropout(0.3) để giảm overfitting.
- Lớp ẩn thứ hai: Dense với 32 nơ-ron, hàm kích hoạt ReLU, theo sau là Dropout(0.3).

- Lớp đầu ra: Dense với 1 nơ-ron và hàm kích hoạt sigmoid để xuất xác suất thuộc lớp dương.

Kiến trúc này cho phép mô hình học các biểu diễn phi tuyến từ dữ liệu, trong khi Dropout giúp giảm overfitting bằng cách vô hiệu hóa ngẫu nhiên 30% nơ-ron trong quá trình huấn luyện.

```
input_dim = X_train.shape[1]

model_dl = models.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')    # nhị phân
])
```

4.3. Compile và cấu hình mô hình

Mô hình được compile với optimizer Adam, hàm mất mát binary_crossentropy (phù hợp với phân loại nhị phân), và metric accuracy để theo dõi hiệu suất trong quá trình huấn luyện. Adam là phương pháp tối ưu thích ứng, phù hợp với dữ liệu lớn và thường hội tụ nhanh hơn các phương pháp truyền thống.

```
model_dl.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])
```

4.4. Compile và cấu hình mô hình

Mô hình được huấn luyện trong 20 epochs với batch size 256. Sử dụng validation_split=0.2 để tự động tách 20% tập huấn luyện làm tập validation, giúp theo dõi hiệu suất và phát hiện overfitting trong quá trình huấn luyện. Tham số verbose=1 cho phép hiển thị tiến trình huấn luyện chi tiết sau mỗi epoch.

```

r = model_dl.fit(
    X_train,
    y_train_dl,
    epochs=20,
    batch_size=256,
    validation_split=0.2,
    verbose=1
)

```

4.5. Đánh giá mô hình trên tập kiểm tra

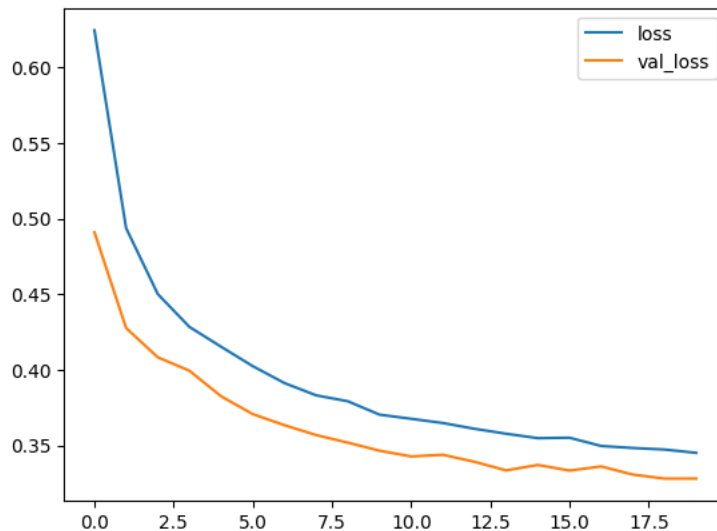
Sau khi huấn luyện, mô hình được đánh giá trên tập kiểm tra. Xác suất dự đoán được lấy từ `model_dl.predict(X_test)` và làm phẳng thành mảng một chiều bằng `.ravel()`. Nhân dự đoán được tạo bằng cách áp dụng ngưỡng 0.5 cho xác suất. Mô hình được đánh giá bằng các chỉ số accuracy, AUC và báo cáo phân loại chi tiết bao gồm precision, recall và F1-score cho từng lớp.

```

# Evaluate test
y_prob = model_dl.predict(X_test).ravel()
y_pred = (y_prob > 0.5).astype(int)

```

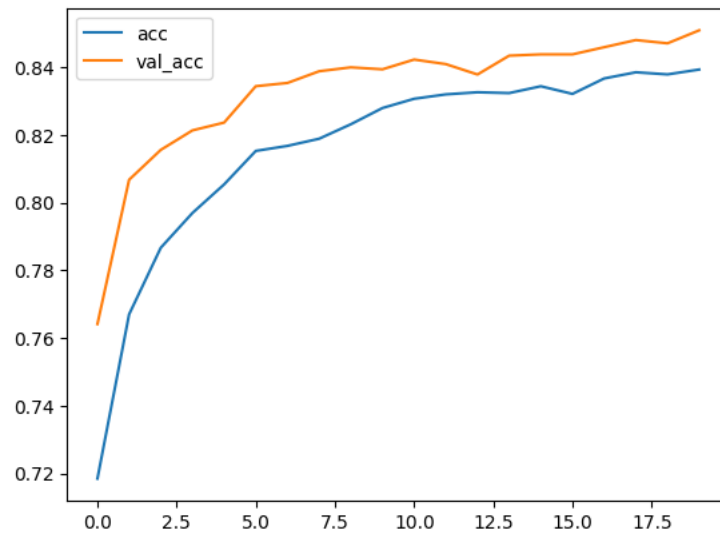
Việc sử dụng mạng neural cho phép mô hình học các mẫu phức tạp và phi tuyến từ dữ liệu, có khả năng cải thiện hiệu suất so với các mô hình truyền thống như Logistic Regression, đặc biệt khi có nhiều tương tác giữa các đặc trưng. Kiến trúc với hai lớp ẩn và dropout giúp mô hình tổng quát hóa tốt trên dữ liệu mới, đồng thời tránh hiện tượng overfitting trong quá trình huấn luyện.



Hình 6

Đường loss train giảm đều từ khoảng 0.62 xuống gần 0.34, điều này cho thấy hàm mất mát trên tập huấn luyện đang được tối ưu dần.

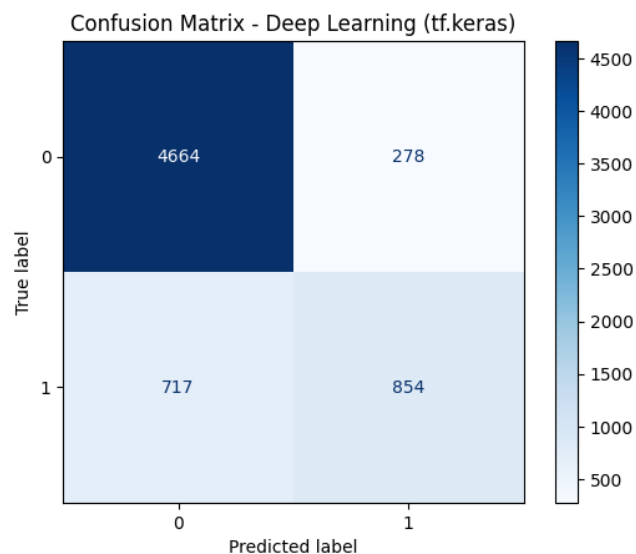
Đường validation cũng giảm và luôn thấp hơn hoặc xấp xỉ loss, cho thấy mô hình không bị overfitting mà còn hơi regularize tốt (validation tốt hơn train).



Hình 7

Đường accuracy tăng dần từ khoảng 0.72 lên gần 0.84, chứng tỏ mô hình ngày càng dự đoán đúng hơn trên tập huấn luyện.

Đường validation luôn cao hơn một chút và cũng tăng ổn định, nghĩa là mô hình tổng quát hóa tốt trên dữ liệu kiểm định, không chỉ “nhớ” dữ liệu train.



Hình 8

V. Đánh giá và so sánh mô hình

5.1. Bảng so sánh hiệu suất các mô hình:

Mô hình	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)	Precision (weighted)	Recall (weighted)	F1-score (weighted)
Logistic Regression	0.8260	0.78	0.70	0.73	0.81	0.83	0.81
Random Forest	0.8609	0.82	0.78	0.80	0.86	0.86	0.86
Tensorflow Keras	0.8386	0.79	0.74	0.76	0.83	0.84	0.83

5.2. Phân tích chi tiết theo lớp:

Mô hình	Lớp	Precision	Recall	F1-score	Support
Logistic Regression	0 (<=50K)	0.85	0.94	0.89	4.942
Logistic Regression	1 (>50K)	0.72	0.46	0.56	1.571
Random Forest	0 (<=50K)	0.89	0.93	0.91	4.942
Random Forest	1 (>50K)	0.75	0.64	0.69	1.571
Tensorflow Keras	0 (<=50K)	0.85	0.95	0.90	4.942
Tensorflow Keras	1 (>50K)	0.76	0.49	0.60	1.571

5.3. Phân tích và so sánh:

5.3.1. Độ chính xác tổng thể:

Về accuracy, Random Forest đạt kết quả cao nhất với khoảng 0.8609, cho thấy mô hình này dự đoán đúng nhiều mẫu hơn so với hai mô hình còn lại trên cùng tập kiểm thử. Tensorflow Keras đạt accuracy khoảng 0.8386, nhỉnh hơn Logistic Regression (0.8260), chứng tỏ mô hình deep learning khai thác tốt hơn các quan hệ phi tuyến trong dữ liệu so với mô hình tuyến tính baseline.

5.3.2. Phân tích theo lớp:

Đối với lớp 0 (thu nhập $\leq 50K$), cả ba mô hình đều đạt F1-score rất cao quanh 0.89–0.91, tức khả năng nhận diện lớp đa số tương đối tương đương. Sự khác biệt rõ rệt nằm ở lớp 1 (thu nhập $> 50K$): Logistic Regression chỉ đạt F1-score 0.56, Tensorflow Keras cải thiện lên 0.60, trong khi Random Forest đạt 0.69 và recall cao nhất (0.64), thể hiện khả năng phát hiện lớp thu nhập cao tốt hơn hẳn.

5.3.3. Chỉ số tổng hợp:

Xét các chỉ số tổng hợp theo macro và weighted, Random Forest vẫn nổi bật với Precision, Recall và F1-score macro lần lượt khoảng 0.82, 0.78, 0.80 và F1-score weighted 0.86, cao hơn Logistic Regression và Tensorflow Keras. Tensorflow Keras đứng thứ hai với F1-score macro 0.76 và weighted 0.83, cho thấy mô hình deep learning cải thiện hiệu năng tổng thể so với Logistic Regression nhưng chưa vượt qua được Random Forest trên bộ dữ liệu Adult.

5.4. Nhận xét về mất cân bằng dữ liệu:

Dữ liệu Adult bị mất cân bằng khá mạnh (lớp 0 chiếm khoảng 76%, lớp 1 chiếm 24%), nên cả ba mô hình đều chịu ảnh hưởng nhất định. Logistic Regression có recall lớp 1 chỉ khoảng 0.46, cho thấy mô hình tuyến tính này vẫn bỏ sót khá nhiều mẫu thu nhập cao do thiên về dự đoán lớp đa số. Random Forest cải thiện đáng kể vấn đề này, với recall lớp 1 tăng lên khoảng 0.64, nhờ cơ chế ensemble nhiều cây quyết định và khả năng học các quan hệ phi tuyến giữa đặc trưng, nên phát hiện lớp thiểu số tốt hơn. Mô hình deep learning triển khai bằng tf.keras cho kết quả nằm giữa Logistic Regression và Random Forest: recall lớp 1 cao hơn Logistic Regression nhưng thường vẫn thấp hơn Random Forest nhẹ, tùy cấu hình mạng và siêu tham số. Điều này cho thấy mạng nơ-ron có khả năng khai thác quan hệ phi tuyến trong dữ liệu và phần nào giảm ảnh hưởng của mất cân bằng lớp, nhưng trên bộ dữ liệu bảng Adult, mô hình ensemble như Random Forest vẫn là lựa chọn hiệu quả và ổn định hơn khi chưa áp dụng thêm các kỹ thuật chuyên biệt cho dữ liệu mất cân bằng (class weight, resampling, focal loss, v.v.).

VI. Kết luận

Trong bài tập lớn này, nhóm đã xây dựng một pipeline học máy hoàn chỉnh cho bài toán dự đoán mức thu nhập trên bộ dữ liệu Adult dạng bảng, bao gồm các bước EDA, tiền xử lý dữ liệu, trích xuất/chuẩn hóa đặc trưng, huấn luyện và đánh giá mô hình. Trên nền tảng đó, ba mô hình chính được triển khai là Logistic Regression (baseline tuyến tính), Random Forest (mô hình ensemble cây quyết định) và mạng nơ-ron deep learning sử dụng tf.keras. Kết quả thực nghiệm cho thấy Logistic Regression cho độ chính xác khá nhưng bị hạn chế ở lớp thu nhập cao; Random Forest đạt hiệu năng tốt nhất với các chỉ số Accuracy, F1-score macro và weighted đều cao, đồng thời cải thiện rõ rệt khả năng nhận diện lớp thu nhập $> 50K$; mô hình deep learning đạt kết quả trung gian, cao hơn Logistic Regression nhưng vẫn kém một chút so với Random Forest trong cấu hình hiện tại.

Từ các thí nghiệm này, nhóm nhận thấy rằng với dữ liệu bảng cỡ vừa và đã được tiền xử lý chuẩn như Adult, các mô hình học máy truyền thống, đặc biệt là các mô hình ensemble như Random Forest, vẫn rất cạnh tranh và thậm chí vượt trội so với mạng nơ-ron sâu đơn giản, đồng thời dễ huấn luyện và diễn giải hơn. Tuy nhiên, việc bổ sung mô hình deep learning giúp mở rộng góc nhìn, cho thấy tiềm năng cải thiện thêm nếu đầu tư vào thiết kế kiến trúc và tinh chỉnh siêu tham số, cũng như nếu trong tương lai áp dụng cho các bộ dữ liệu lớn và phức tạp hơn.

References

Dữ liệu (Datasets)

- UCI Machine Learning Repository (n.d.). Adult (Census Income) Dataset. University of California, Irvine. Truy cập từ: <http://archive.ics.uci.edu/ml/datasets/adult>
- Kaggle (n.d.). Adult Census Income / UCI Adult Census Data Dataset.

Bài viết/Hướng dẫn Trực tuyến

- DigitalOcean (n.d.). Logistic Regression with Scikit-learn. Truy cập từ: <https://www.digitalocean.com/community/tutorials/logistic-regression-with-scikit-learn>
- GeeksforGeeks (n.d.). Random Forest Classifier Using Scikit-learn. Truy cập từ: <https://www.geeksforgeeks.org/dsa/random-forest-classifier-using-scikit-learn/>
- Viblo (n.d.). Logistic Regression - Bài toán cơ bản trong Machine Learning. Truy cập từ: <https://viblo.asia/p/logistic-regression-bai-toan-co-ban-trong-machine-learning-924lJ4rzKPM>
- deeplearningcourses.com (n.d.). Notebooks. Truy cập từ: <https://deeplearningcourses.com/notebooks/3TJiBypUfGKrDmo2rx51Lg>